

Optimization over Trained Neural Networks: Going Large with Gradient-Based Algorithms

Jiatai Tong[1], Yilin Zhu[2], Thiago Serra[2(✉)], and Samuel Burer[2]

[1] Northwestern University, Evanston, IL, USA
jiataitong2026@u.northwestern.edu
[2] University of Iowa, Iowa City, IA, USA
{yilin-zhu,thiago-serra,samuel-burer}@uiowa.edu

Abstract. When optimizing a nonlinear objective, one can employ a neural network as a surrogate for the nonlinear function. However, the resulting optimization model can be time-consuming to solve globally with exact methods. As a result, local search that exploits the neural-network structure has been employed to find good solutions within a reasonable time limit. For such methods, a lower per-iteration cost is advantageous when solving larger models. The contribution of this paper is two-fold. First, we propose a gradient-based algorithm with lower per-iteration cost than existing methods. Second, we further adapt this algorithm to exploit the piecewise-linear structure of neural networks that use Rectified Linear Units (ReLUs). In line with prior research, our methods become competitive with—and then dominant over—other local search methods as the optimization models become larger.

Keywords: Constraining learning · Gradient ascent · Linear regions · Piecewise-linear functions · Rectified linear units

1 Introduction

In the field of mathematical programming, piecewise-linear functions play an important role in modeling nonlinear functions [25,34,36,37,60]. In deep learning, a popular model that provides a piecewise-linear approximation of a nonlinear function is the neural network with the ReLU activation function [2,24]. Researchers have long known that some neural network architectures are universal function approximators [11,15,22], and in particular, this is also true of the ReLU activation function if the architecture is sufficiently wide [62] or deep [20]. When neural-network approximations are used as surrogates for solving nonlinear optimization problems, algorithms that exploit the piecewise-linear structure of the neural networks are of particular interest. In this paper, we propose gradient-based algorithms for this setting.

Optimizing a piecewise-linear function over a polyhedron can be modeled using a Mixed-Integer Linear Programming (MILP) formulation. In the specific

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
T. Guns (Ed.): CPAIOR 2026, LNCS 16595, pp. 596–613, 2026.
https://doi.org/10.1007/978-3-032-27242-3_35

case of ReLU networks, existing MILP formulations either have a weak linear relaxation due to big M coefficients [14] or become prohibitively large when using a disjunctive formulation [23]. Researchers have found success by improving the big M coefficients [3,10,16,21,30,51,63], strengthening formulations using valid inequalities [1], and using a hybrid of both formulations [56]. Other improvements include reformulation [23,32,45], parameter rescaling [40], pruning the search space by inference [6,46,54,61], and working with sparser neural networks [7,39, 44,61]. Those improvements can also help using MILP formulations in neural networks for verification [1,10,43,52], compression [48,49], evaluating output variation [8,29,31,46,47], and counterfactual explanations [28,57].

There are also local search methods, which do not solve MILPs exactly, but instead are designed to find good solutions in limited time. Indeed, both Perakis & Tsiourvas [38] and Tong et al. [55] are closely related to this paper for taking a geometric view of the input space of the ReLU network, focusing on the linear pieces of the function approximation. Known as *linear regions* in machine learning, each piece corresponds to a polyhedron associated with a distinct set of active neurons. Within a linear region, changes to the input have a linear impact on the output. In Fig. 1, we illustrate these concepts on a neural network having inputs x_1 and x_2, neurons with outputs h_1 to h_5, and output y.

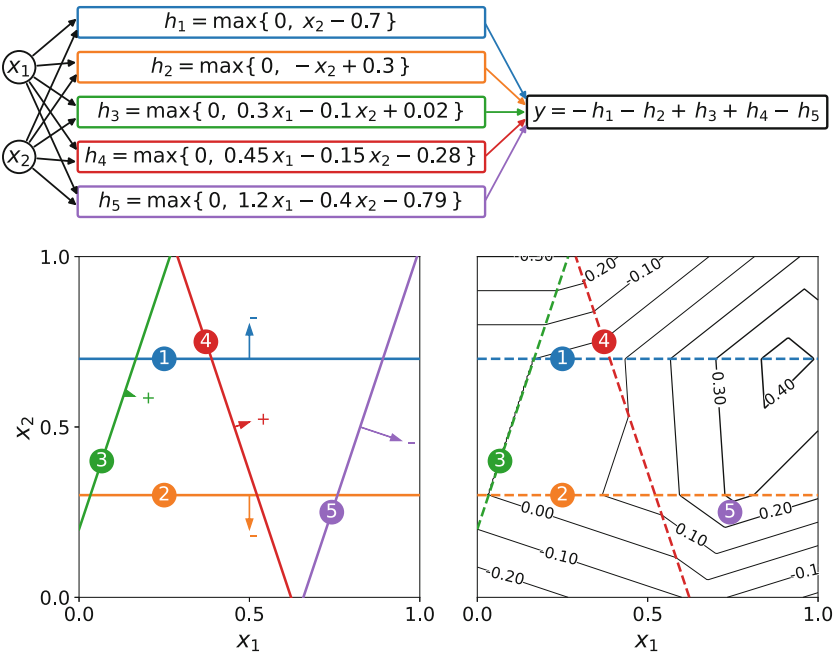


Fig. 1. Top: Visual description of neural network used as example. **Bottom left:** Lines partitioning the space based on what inputs produce a positive output for each neuron, with the arrow pointing to the positive side, the length of the arrow proportional to the magnitude of the parameters, and the arrow label denoting the influence on y. **Bottom right:** Contour plots of y over the lines associated with neural activations.

We summarize the works [38,55] just mentioned, taking the liberty to name them *MILP Walk* and *LP Walk*, respectively, in order to draw parallels between these methods and our methods introduced in Sect. 3:

- **MILP Walk:** Perakis & Tsiourvas [38] solve a sequence of restricted MILP models. Each MILP model finds the best solution across all linear regions containing the current solution. If a better solution is found, the same process is repeated from the new solution. In Fig. 2 Left, solution A lies only in the linear region in darkest gray, in which the best solution is B. In turn, solution B lies in the four linear regions with the three darker tones of gray, where the best solution is C. Finally, solution C lies in the four linear regions with the two lighter tones of gray, where the best solution is C again. Once no improvement is found, the algorithm stops.
- **LP Walk:** Tong et al. [55] solve a sequence of LP models. Each LP model finds the best solution in a linear region containing the current solution. If a better solution is found, they repeat the process by moving slightly past the new solution along the line from the last solution. Moving slightly past avoids using a solution lying in multiple linear regions. In Fig. 2 Right, the linear region of solution A is in darker gray, and its best solution is along the line from A to B. In turn, the linear region of solution B is in slightly lighter gray, and its the best solution is along the line from B to C. From C we find solution E and move towards D. From D we find solution E again. At this point, the algorithm stops.

For a broader discussion about linear regions and their nexus with mathematical optimization, we recommend the survey by Huchette et al. [24].

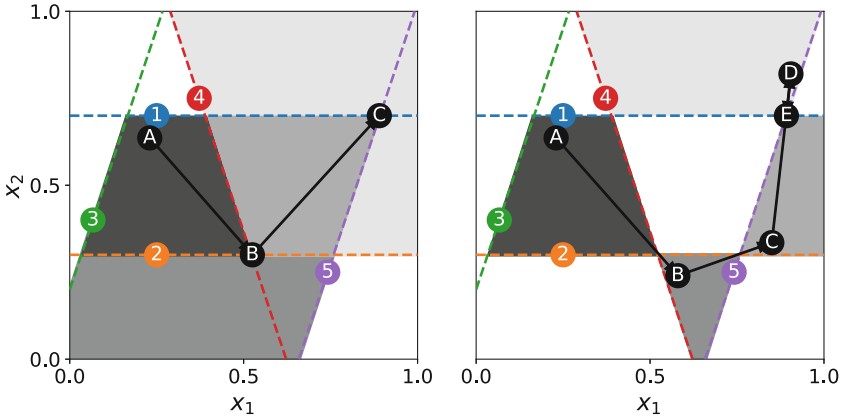


Fig. 2. Left: Solutions found by MILP Walk from the initial solution $A = (0.23, 0.636)$ until convergence. **Right:** Solutions found by LP Walk from A until convergence.

Which algorithm, MILP Walk or LP Walk, is best suited for a particular instance often depends on the size of that instance. Starting from the same

solution, it is easy to see that MILP Walk will move next to a solution that is at least as good as the one found by LP Walk. On the other hand, each iteration of MILP Walk solves the same MILP model used to optimize over the entire neural network, albeit restricted to the neighborhood of the current solution. Hence, the per-iteration cost of MILP Walk is higher than the per-iteration cost of LP Walk. Consequently, LP Walk can perform more iterations in the same amount of time. Indeed, an empirical comparison of both methods shows that LP Walk performs better for neural networks with more inputs, layers, and neurons [55], all of which imply a larger number of linear regions [47]. Thus, LP Walk conducts a style of search that is more akin to sampling than to enumeration [46].

Of course, solving an LP model for each linear region, as in LP Walk, may eventually become too costly in ever larger neural networks. Hence, in this paper, we propose a new local search approach with an even smaller per-iteration cost:

– **Gradient Walk:** We compute a sequence of gradient steps. Each step may find a better solution within the current linear region, or a solution in another linear region that is better or worse. We keep track of the best solution found. If the improvement is too small for a predefined number of steps, we restart from a perturbation of the best solution found thus far. In Fig. 3 Left, we move from A to B through nine intermediary steps in the same linear region as A, all in the same direction, and then from B to C with five intermediary steps. Note that the steps are orthogonal to the contour plots within the linear regions. In Fig. 3 Right, we continue from C until H by zig-zagging among linear regions and improving in all steps except G. The steps following H would not find a much better solution. Hence, the algorithm stops.

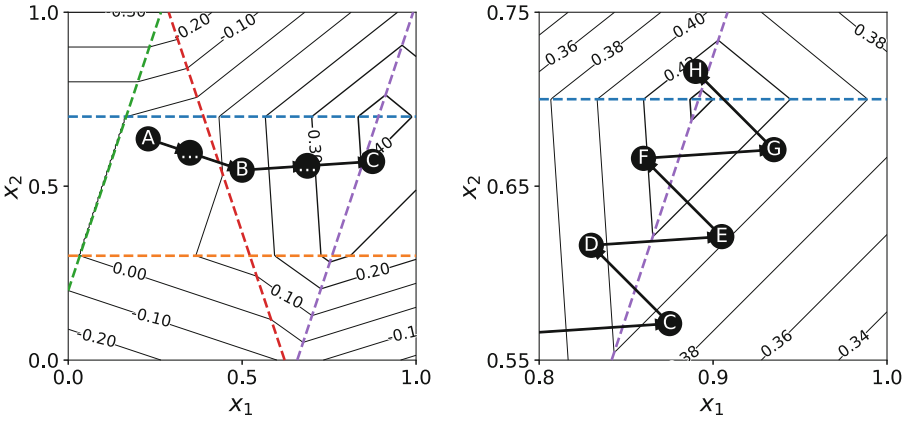


Fig. 3. Left: First solutions found by Gradient Walk from the initial solution $A = (0.23, 0.636)$. **Right:** Next solutions found over a narrower region of the input space.

In what follows, we define our problem of interest and its conventional MILP model in Sect. 2. Then we present an algorithm for Gradient Walk as well as a

variant that further exploits knowledge of linear regions in Sect. 3. We evaluate those algorithms in Sect. 4. Conclusions are given in Sect. 5.

2 Preliminaries

We optimize the function $f : \mathbb{R}^{n_0} \to \mathbb{R}$ associated with a neural network over a polytope $\mathbb{X} \subset \mathbb{R}^{n_0}$:

$$\text{maximize}_{\boldsymbol{x}} \quad f(\boldsymbol{x}) \tag{1}$$
$$\text{s.t.} \quad \boldsymbol{x} \in \mathbb{X} \tag{2}$$

i.e., we want an input $\boldsymbol{x} = [x_1\ x_2\ \ldots\ x_{n_0}]^\top \in \mathbb{X}$ maximizing the prediction $f(\boldsymbol{x})$. In the experiments of Sect. 4, $\mathbb{X}$ equals a box, but our theoretical development requires only that $\mathbb{X}$ be polyhedral.

Let the neural network have L hidden layers, each hidden layer $l \in \mathbb{L} := \{1, \ldots, L\}$ having preactivation values $\boldsymbol{g}^l = [g_1^l\ g_2^l \ldots g_{n_l}^l]^\top$ and outputs $\boldsymbol{h}^l = [h_1^l\ h_2^l \ldots h_{n_l}^l]^\top$ from neurons indexed by $i \in \mathbb{N}_l = \{1, 2, \ldots, n_l\}$. The output layer $L+1$ has a single preactivation value g_1^{L+1} as the network output. Let $\boldsymbol{W}^l$ and $\boldsymbol{b}^l$ denote the weight matrix and bias vector associated with the l-th layer, for which we assume that all elements are within $[-1, 1]$. The preactivation value g_i^l of neuron i in layer l is given by $g_i^l = \boldsymbol{W}_i^l \boldsymbol{h}^{l-1} + \boldsymbol{b}_i^l$, and the output h_i^l of neuron i in hidden layer l follows by the ReLU activation $h_i^l = \max\{0, g_i^l\}$. In this setting, $\boldsymbol{x} = \boldsymbol{h}^0$ and $f(\boldsymbol{x}) = g_1^{L+1}$.

For an input $\boldsymbol{x} \in \mathbb{X}$, let $\boldsymbol{z}^l(\boldsymbol{x}) = [z_1^l\ z_2^l \ldots z_{n_l}^l]^\top$ be the *layer activation pattern* produced in layer l of the neural network when given $\boldsymbol{x}$ as input, where:

$$z_i^l = \begin{cases} 1, & \text{if } h_i^l = g_i^l \geq 0, \\ 0, & \text{if } g_i^l \leq 0. \end{cases} \tag{3}$$

A neuron is *binding* if $g_i^l = h_i^l = 0$. In this case, z_i^l can be either 0 or 1.

Let $\boldsymbol{z}(\boldsymbol{x}) = \{\boldsymbol{z}^1(\boldsymbol{x}), \boldsymbol{z}^2(\boldsymbol{x}), \ldots, \boldsymbol{z}^L(\boldsymbol{x})\}$ be the *activation pattern* produced across all layers of the neural network when given $\boldsymbol{x}$ as input. A *linear region* $\mathcal{R}_{\boldsymbol{z}'} \in \mathbb{X}$ is a set where every input $\boldsymbol{x}$ has the same activation pattern $\boldsymbol{z}'$ as all the other inputs, i.e., $\boldsymbol{z}(\boldsymbol{x}) = \boldsymbol{z}'\ \forall \boldsymbol{x} \in \mathcal{R}_{\boldsymbol{z}'}$. The output of f varies linearly within each linear region. Inputs lie in multiple linear regions if a neuron is binding.

We can formulate the optimization problem as the following MILP model:

$$\text{maximize} \quad f(\boldsymbol{x}) = g_1^{L+1} \tag{4}$$
$$\text{s.t.} \quad \boldsymbol{h}^0 = \boldsymbol{x}, \qquad \boldsymbol{x} \in \mathbb{X} \tag{5}$$
$$\boldsymbol{W}_i^l \boldsymbol{h}^{l-1} + \boldsymbol{b}_i^l = \boldsymbol{g}_i^l, \qquad \forall l \in \mathbb{L} \cup \{L+1\}, i \in \mathbb{N}_l \tag{6}$$
$$(\boldsymbol{z}_i^l = 1) \to (\boldsymbol{h}_i^l = \boldsymbol{g}_i^l), \qquad \forall l \in \mathbb{L}, i \in \mathbb{N}_l \tag{7}$$
$$(\boldsymbol{z}_i^l = 0) \to (\boldsymbol{g}_i^l \leq 0 \wedge \boldsymbol{h}_i^l = 0), \qquad \forall l \in \mathbb{L}, i \in \mathbb{N}_l \tag{8}$$
$$\boldsymbol{h}_i^l \geq 0, \qquad \boldsymbol{z}_i^l \in \{0, 1\}, \qquad \forall l \in \mathbb{L}, i \in \mathbb{N}_l \tag{9}$$

The indicator constraints (7)–(8) can be modeled with big M constraints [5].

We can formulate an LP model optimizing over a single linear region $\mathcal{R}_{\boldsymbol{z}'}$ by fixing $\boldsymbol{z} = \boldsymbol{z}'$. For example, this is what LP Walk does in each iteration.

3 Our Proposed Algorithms

We propose two gradient-based algorithms for solving (1)–(2). We assume throughout that the polyhedral domain $\mathbb{X}$ is bounded, i.e., $\mathbb{X}$ is a polytope. Indeed, neural networks are often used within a confined domain. There is a growing body of work on preventing a neural network from extrapolating outside the nominal or implied domain defined by a training set [50,57,64]. Moreover, the possibility of embedding a neural network as part of an MILP model depends on the input set being bounded [47], which otherwise would also prevent us from benchmarking against existing algorithms [38,55].

Our first algorithm, the Perturbed Projected Gradient Ascent (`PPGA`) algorithm, is described in Sect. 3.1. `PPGA` makes use of both the piecewise-constant nature of ∇f and projection onto $\mathbb{X}$. Our second algorithm, `PPGA` with Linear Region Valve ($\texttt{PPGA}_{\text{LR}}$), is described in Sect. 3.2. $\texttt{PPGA}_{\text{LR}}$ enhances `PPGA` by further exploiting the structure of the linear regions, being particularly beneficial if the number of linear regions is large.

3.1 Perturbed Projected Gradient Ascent (Algorithm 1)

We first mention the standard approach, called the Projected Gradient Ascent (`PGA`) algorithm, of projecting the gradient steps over the feasible set to produce iterates of the form

$$\boldsymbol{x}^{t+1} = P_{\mathbb{X}}(\boldsymbol{x}^t + \gamma \nabla f(\boldsymbol{x}^t)), \tag{10}$$

where γ is the learning rate and L_2 projection [41,42] solves the convex quadratic program (QP)

$$P_{\mathbb{X}}(\dot{\boldsymbol{x}}) = \arg\min_{\boldsymbol{x}} \quad \|\boldsymbol{x} - \dot{\boldsymbol{x}}\|^2 \tag{11}$$

$$\text{s.t.} \quad \boldsymbol{x} \in \mathbb{X}, \tag{12}$$

which takes only $O(n_0)$ time when $\mathbb{X}$ is a box.

Based on `PGA`, we now introduce our method `PPGA`. For any point $\boldsymbol{x}' \in \mathcal{R}_{\boldsymbol{z}(\boldsymbol{x})}$, i.e., any point $\boldsymbol{x}'$ in the same linear region as $\boldsymbol{x}$, we can calculate $f(\boldsymbol{x}')$ with the affine transformation

$$f|_{\mathcal{R}_z(\boldsymbol{x})}(\boldsymbol{x}') = T(\boldsymbol{z}(\boldsymbol{x}))\boldsymbol{x}' + t(\boldsymbol{z}(\boldsymbol{x})), \tag{13}$$

where

$$T(\boldsymbol{z}(\boldsymbol{x})) = \boldsymbol{w}^{L+1} \left(\prod_{l=1}^{L} (\boldsymbol{z}^l(\boldsymbol{x})\boldsymbol{I})\boldsymbol{W}^l \right) \tag{14}$$

and $t(\boldsymbol{z}(\boldsymbol{x}))$ is constant [24]. Hence, it follows that

$$\nabla f(\boldsymbol{x}) = \boldsymbol{w}^{L+1} \left(\prod_{l=1}^{L} (\boldsymbol{z}^l \boldsymbol{I}) \boldsymbol{W}^l \right) \tag{15}$$

for any point $\boldsymbol{x}$ at the interior of its linear region, i.e., not binding for any neuron. Hence, f has a piecewise affine landscape, which may contain local maxima, saddle points, and local minima.

For smooth maximization problems, one may generally escape from (interior feasible) saddle points and local minima by introducing a perturbation when $\nabla f(\boldsymbol{x})$ is small enough [17,26,27,59]. Our case is quite different, however, since all differentiable regions have constant gradients and because saddle points and local minima occur at the boundary of those regions, where the function is

Algorithm 1. Perturbed Projected Gradient Ascent

Input: Model f with input size n_0, learning rate γ, time limit T, restart noise coefficient Ξ, error threshold ϵ, tolerance window k; input domain $\mathbb{X}$
Output: Best solution $\boldsymbol{x}^*$ and objective value $f(\boldsymbol{x}^*)$

$\delta \leftarrow \frac{\Xi}{\sqrt{n_0}}$
Sample initial $\boldsymbol{x} \sim \text{Uniform}(\mathbb{X})$
$\boldsymbol{x}^* \leftarrow \boldsymbol{x}$ ▷ Initialized best solution so far
$\boldsymbol{x}' \leftarrow \boldsymbol{x}$ ▷ Initialized best solution since reset
$r \leftarrow 0$ ▷ Initialized reset counter
while Time $< T$ **do**
  $\boldsymbol{x} \leftarrow P_{\mathbb{X}}(\boldsymbol{x} + \gamma \nabla f(\boldsymbol{x}))$ ▷ Gradient step; replaced with Algorithm 2 in $\texttt{PPGA}_{\text{LR}}$
  if $f(\boldsymbol{x}) > f(\boldsymbol{x}')$ **then** ▷ If found best solution since reset
    $\Delta \leftarrow f(\boldsymbol{x}) - f(\boldsymbol{x}')$ ▷ Calculate local improvement before update
    $\boldsymbol{x}' \leftarrow \boldsymbol{x}$ ▷ Update best solution since reset
    if $f(\boldsymbol{x}) > f(\boldsymbol{x}^*)$ **then** ▷ If found best overall solution
      $\boldsymbol{x}^* \leftarrow \boldsymbol{x}$ ▷ Update best overall solution
    end if
    if $\Delta < f(\boldsymbol{x}) \cdot \epsilon$ **then** ▷ If local improvement is below threshold
      $r \leftarrow r + 1$ ▷ Increment reset counter
      if $r = k$ **then** ▷ If reached reset trigger
        $\boldsymbol{x} \leftarrow P_{\mathbb{X}}(\boldsymbol{x}^* + \xi), \quad \xi \sim \mathcal{N}(0, \delta)$ ▷ Reset event
        $\boldsymbol{x}' \leftarrow \boldsymbol{x}$
        $r \leftarrow 0$
      end if
    else ▷ If the improvement is above the threshold
      if $f(\boldsymbol{x}) = f(\boldsymbol{x}^*)$ **then** ▷ If current solution matches best overall solution
        $r \leftarrow 0$ ▷ Reset the counter
      end if
    end if
  end if
end while
return $\boldsymbol{x}^*$, $f(\boldsymbol{x}^*)$

nondifferentiable. Hence, the size of $\nabla f(x)$ is not a reliable measure of local optimality in our case.

We hence use a different mechanism to measure progress. If we accumulate k improvements that are relatively small in comparison to the current objective value $f(x^t)$, while not producing a better overall solution, then we continue the next iteration from a perturbation $\xi \sim \mathcal{N}(0, \frac{\bar{\varepsilon}}{\sqrt{n_0}})$ around the current best solution x^*. If that happens at step t, then the next update is

$$x^{t+1} = P_{\mathbb{X}}(x^* + \xi + \gamma \nabla f(x^* + \xi)) \tag{16}$$

The resultant algorithm, our Perturbed Projected Gradient Ascent (`PPGA`) algorithm, is described as Algorithm 1.

3.2 `PPGA` With Linear Region Valve (Algorithms 1 and 2)

When the ReLU network gets deeper, the gradient calculated with Equation (15) may either explode or diminish, which makes it important to find an appropriate learning rate. As an alternative to calibrate the learning rate, we propose using linear region information for making local decisions about the size of the gradient step. Because we assume in this paper that all weights are within $[-1, 1]$, we expect the gradient to diminish when the network gets deeper. Notably, a similar approach can also be applied to resolve gradient explosion.

Suppose that we are in the linear region $\mathcal{R}_{\boldsymbol{z}}$ with activation pattern $\boldsymbol{z}$. We can calculate how far we may move to reach the next linear region in the direction of the gradient by a ratio test:

$$u = \min_{i \in \mathcal{I}} \left(-\frac{g_i}{\Delta g_i(\boldsymbol{x})}\right) \tag{17}$$

where $\mathcal{I} = \left\{ i \mid i \in \mathbb{N}_l, l \in \mathbb{L}, -\frac{g_i}{\Delta g_i(\boldsymbol{x})} \geq 0 \right\}$ is a subset of all neurons, and

$$\Delta g_i(\boldsymbol{x}) = g_i(\boldsymbol{x} + \nabla f(\boldsymbol{x})) - g_i(\boldsymbol{x}). \tag{18}$$

Here, u is the relative step size to the next linear region, while the actual step size is $u \cdot \|\nabla f(\boldsymbol{x}^t)\|$. We can use u as an estimate for the size of a linear region $\mathcal{R}_{\boldsymbol{z}'}$ near $\mathcal{R}_{\boldsymbol{z}}$, i.e., $\|\boldsymbol{z} - \boldsymbol{z}'\|_1 \leq \zeta$ with a relatively small ζ. Given also the relative step size γ, then we estimate the gradient step to stretch over

$$v = \left\lceil \frac{\gamma}{u} \right\rceil \tag{19}$$

linear regions around $\mathcal{R}_{\boldsymbol{z}}$.

Let $V > 1$ be a predetermined valve value, which corresponds to the number of linear regions that we would like to stretch over at each gradient step. If γ is such that $v < V$ at the current linear region, then we use a scale factor c over the magnitude of the gradient:

$$\boldsymbol{x}^{t+1} = \begin{cases} P_{\mathbb{X}}\left(\boldsymbol{x}^t + c\,\dfrac{\nabla f(\boldsymbol{x}^t)}{\|\nabla f(\boldsymbol{x}^t)\|}\right), & \text{if } \gamma \leq V \cdot u, \\ P_{\mathbb{X}}\left(\boldsymbol{x}^t + \gamma\,\nabla f(\boldsymbol{x}^t)\right), & \text{otherwise.} \end{cases} \tag{20}$$

Algorithm 2. Adaptive Gradient Step with Linear Region Valve

Input: Current solution $\boldsymbol{x}$, learning rate γ, valve value V, scale factor c
Output: New iterate $\boldsymbol{x}'$ ▷ Replaces iterate calculated in Line 7 of Algorithm 1

1: $\nabla f(\boldsymbol{x}) \leftarrow \boldsymbol{w}^{L+1}\left(\prod_{l=1}^{L}(\boldsymbol{z}^l \boldsymbol{I})\boldsymbol{W}^l\right)$
2: $g \leftarrow f(\boldsymbol{x})$
3: $g' \leftarrow f\big(\boldsymbol{x} + \nabla f(\boldsymbol{x})\big)$
4: $\Delta g \leftarrow g' - g$
5: $\rho \leftarrow -g/\Delta g$
6: Mask all negative entries in ρ with $+\infty$
7: $u \leftarrow \min(\rho)$
8: **if** $V \cdot u \geq \gamma$ **then**
9: $\quad \boldsymbol{x}' \leftarrow P_{\mathbb{X}}\left(\boldsymbol{x} + c \cdot \dfrac{\nabla f(\boldsymbol{x})}{\|\nabla f(\boldsymbol{x})\|}\right)$
10: **else**
11: $\quad \boldsymbol{x}' \leftarrow P_{\mathbb{X}}\big(\boldsymbol{x} + \gamma \cdot \nabla f(\boldsymbol{x})\big)$
12: **end if**
13: **return** $\boldsymbol{x}'$

The adaptive gradient step replaces Line 7 in Algorithm 1 with Algorithm 2.

In our implementation, we simply chose $V = \frac{1}{\|\nabla f(\boldsymbol{x}^t)\|}$ as an adaptive valve value, so that a small gradient will more likely trigger the rescaled gradient update. We also set $c = u$, so that we force the step size stretching more than V linear regions, since each linear region is estimated to have length $u \cdot \|\nabla f(\boldsymbol{x}^t)\| = \frac{u}{V}$. The main reason using adaptive hyperparameters is that applying grid search to extra hyperparameters is very costly, and we want to be fair to those algorithms with fewer hyperparameters, such as `PPGA` and [55].

We denote this variant of `PPGA` using Algorithm 2 and adaptive (V, c) design as $\texttt{PPGA}_{\text{LR}}$.

4 Numerical Experiments

We devised numerical experiments to evaluate algorithms `PPGA` and $\texttt{PPGA}_{\text{LR}}$ on standard benchmarks and compare them with other methods. In the following subsections, we define the concept of a basic experiment, then describe our method for generating multiple experiments, and finally detail the optimization results. All numerical experiments were implemented in Python 3.10.8 using Gurobi 11.0 and evaluated on a single Xeon E5-2680v4 core running at 2.4 GHz with 16 GB of memory under the CentOS Linux operating system. The source code is publicly shared at https://github.com/yillzhu/nn_opt.

4.1 Definition of an Experiment

In our study, an *experiment* $\mathcal{P}$ refers to a complete specification of five design options that determine the structure of a neural network and the algorithm used:

- *Input size*, i.e., the input dimension n_0 of the neural network.
- *Depth*, i.e., the number of hidden layers d of the neural network.
- *Width*, i.e., the number of neurons m in each hidden layer.
- *Seed*, i.e., the seed s used to instantiate the network parameters.
- *Algorithm*, i.e., the local search method $\mathcal{M}$ used for optimization.

Given a particular combination of these specifications, the experiment proceeds as follows. With seed s fixed, we generate a neural network having input dimension n_0, depth d, and width m. For each optimization algorithm $\mathcal{M}$, we then conduct a grid search to determine the optimal hyperparameters: shaking noise σ, error threshold ϵ, and tolerance window k. Once these parameters are fixed, the algorithm $\mathcal{M}$ is used to optimize the network within a specified time limit.

In line with prior work [38,55] and to properly benchmark with it, we optimize over neural networks with their weights as defined at initialization. Hence, instead of optimizing over neural networks approximating specific functions based on their training, we work with neural networks representing distinct and random functions. We believe that this makes the results more representative.

4.2 Generating Multiple Experiments

In addition to `PPGA` and $\texttt{PPGA}_{\text{LR}}$, we use the `PGA` algorithm from Sect. 3.1 as a baseline and benchmark against the algorithm proposed for LP Walk in [55], which we denote as `SimplexWalk`. `SimplexWalk` has shown better scalability than solving directly with Gurobi [19] or with MILP Walk [38]. Hence,

$$\mathcal{M} \in \{\texttt{PPGA}_{\text{LR}}, \texttt{PPGA}, \texttt{PGA}, \texttt{SimplexWalk}\}\,.$$

We use input size $n_0 \in \{10, 100, 1000\}$, depth $d \in \{2, 4, 6, 8\}$, and width $m \in \{100, 1000, 10000\}$, resulting in $3 \times 3 \times 4 = 36$ distinct network configurations.

For gradient-based algorithms ($\texttt{PPGA}_{\text{LR}}$, `PPGA`, `PGA`), performance is very susceptible to hyperparameters. Therefore, we use a preprocessing phase to search for better hyperparameters for each gradient-based algorithm through grid search and voting. For each network configuration (n_0, d, m), we generate five random networks using seeds $s \in \{5, 6, 7, 8, 9\}$. These networks, denoted as *grid search instances*, are used for choosing hyperparameters over a grid with

$$\gamma \in \{0.001, 0.01, 0.1, 1, 5\}, \quad \sigma \in \{0.2, 2, 20\}, \quad k \in \{100, 500, 1000\}.$$

For each grid search instance, we evaluate all $5 \times 3 \times 3 = 45$ parameter combinations, running the optimization for 5 min per grid point. The five combinations achieving the best objective values are recorded for each instance. After processing all five grid search instances, we obtain five sets of top-performing parameter combinations. The most frequently occurring combination across these sets is selected as the final grid search result for that algorithm and network configuration. This process runs independently for each gradient-based algorithm.

Once the hyperparameters are calibrated, we generate 20 additional networks for each (n_0, d, m) combination using seeds $s \in \{10, 11, \ldots, 29\}$. These are

referred to as *optimization instances*. Each algorithm $\mathcal{M}$ is then applied to these instances, and the gradient-based algorithms use the hyperparameters obtained from the grid search. Every run is executed for 7200 s, during which we record, at every second, the best objective value and the number of iterations.

4.3 Computational Complexity of Walk Steps

The complexity of the problem can be affected by all three setup options (n_0, d, m). For gradient-based algorithms (`PPGA`$_{\text{LR}}$, `PPGA`, `PGA`), the asymptotic complexity of both calculating the gradient and making a prediction is

$$O(dm^2 + n_0 m).$$

which can be deduced from a sequence of vector-matrix multiplications. Through Eq. (18), each step of the linear region algorithm (`PPGA`$_{\text{LR}}$) may cost twice as much as `PGA` and `PPGA` steps, which we may consider still acceptable. On the other hand, `SimplexWalk` carries out the solution of one LP model per step. Notably, the computational cost of a `SimplexWalk` step, if the LP model is solved using the simplex algorithm, is exponential on n_0 in worst-case.

4.4 Results

We use the Dolan–More performance profile [12,53] to compare the performance of different algorithms on multiple problems. A performance profile shows, for each algorithm, the fraction of test instances on which it performs within a given factor of the best observed result. The horizontal axis represents the performance factor $\tau \geq 1$ in log scale, and the vertical axis $\rho_{\mathcal{M}}(\tau) \in [0, 1]$ represents the fraction of experiments for which the algorithm $\mathcal{M}$ attains a performance ratio of at most τ. The vertical intercept $\rho_{\mathcal{M}}(1)$ indicates how often an algorithm achieves the best result among all competitors, and a curve approaching $\rho = 1$ more rapidly reflects an algorithm whose performance is consistently close to the best algorithm across all experiments.

Figure 4 shows the overall performance profiles across all setups after the methods have been run for 30, 60, and 120 min. The other two plots focuses on the results after 120 min. Figure 5 shows the performance profiles from partitioning the instances according to the input size of the neural network. Figure 6 partitions the instances according to the depth and the width of the neural network.

4.5 Analysis

In general, the relationship between the three gradient algorithms is similar across different time budgets, with `PPGA` and `PPGA`$_{\text{LR}}$ outperforming the vanilla `PGA`. However, the gap between the first `PPGA` and vanilla `PGA` shrinks for the largest width (10000). Since width dominates the complexity of gradient steps, it is possible that `PPGA` algorithms do not gain much in performance advantage

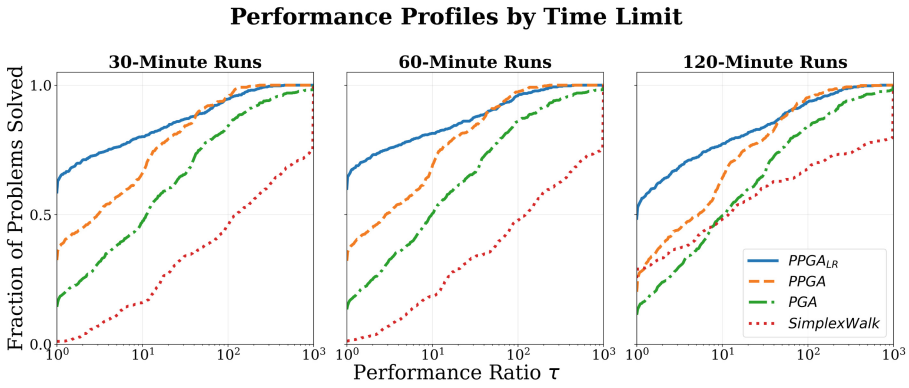


Fig. 4. Comparison of algorithm performance over all instances by varying time limit.

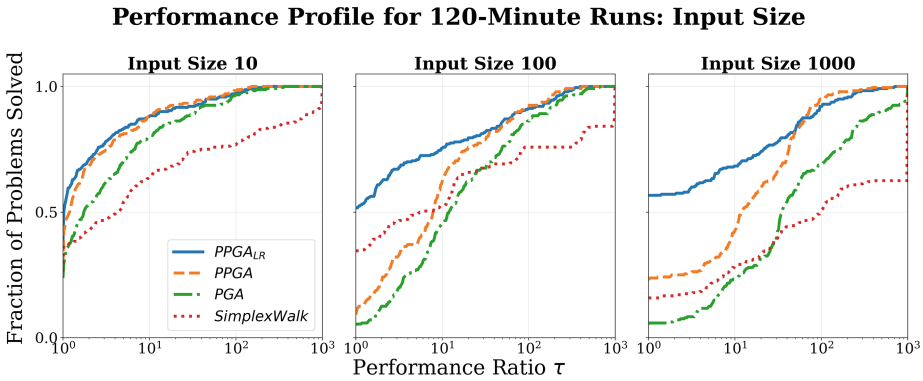


Fig. 5. Comparison of algorithm performance by varying input size in 120-minute runs.

in the early steps compared to the vanilla algorithm. Therefore, the advantage that we observe is likely gained in the later steps, when the vanilla algorithm is captured in some local optimum while PPGA escapes through perturbation.

While $\texttt{PPGA}_{\text{LR}}$ is overall better than PPGA in the aggregate of instances from Fig. 4, the advantage of $\texttt{PPGA}_{\text{LR}}$ is due to the instances with larger dimensions:

- From Fig. 5, we see that $\texttt{PPGA}_{\text{LR}}$ matches PPGA for the smallest input (10) but does significantly better than PPGA for larger input sizes (100 and 1000).
- From the rows of Fig. 6, $\texttt{PPGA}_{\text{LR}}$ is generally better for larger widths.
- From the columns of Fig. 6, we see that $\texttt{PPGA}_{\text{LR}}$ is worse for the smallest depth (2), but that it dominates the results for deeper networks (6 and 8).

Hence, larger input size and depth seem to be the most determinant factors for $\texttt{PPGA}_{\text{LR}}$ performing better than PPGA. Larger width also contributes in some cases. Those conditions conform with the cases in which neural networks tend

to have more linear regions, which has inspired the design of $\texttt{PPGA}_{\text{LR}}$ in the first place. Moreover, as indicated by Eq. 15 and given the weight distribution, neural networks with greater depth are bound to have smaller gradients.

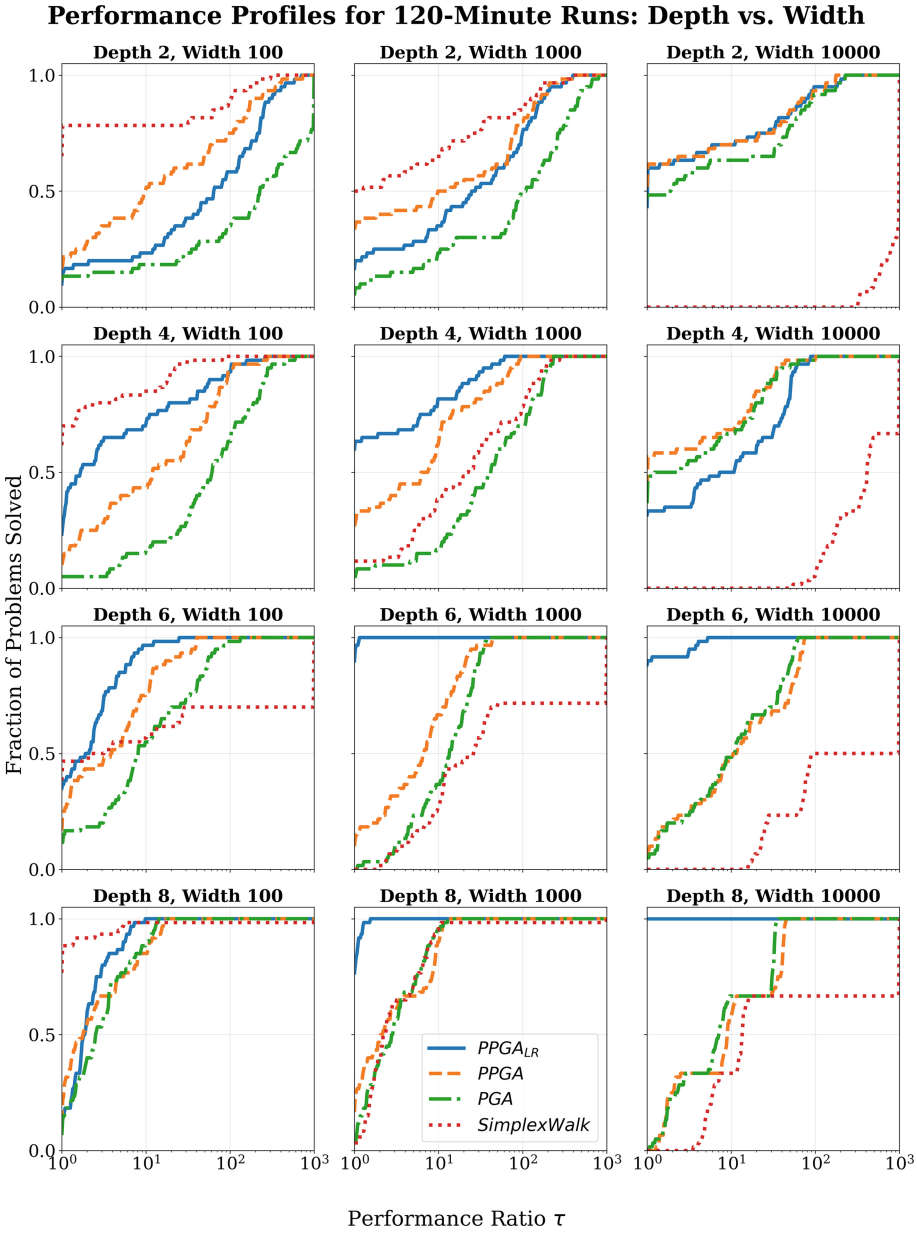


Fig. 6. Comparison of performance by varying depth and width in 120-minute runs.

Meanwhile, notably, `SimplexWalk` gets comparatively better with longer runtimes, as shown in Fig. 4. That conforms with the intuition that it takes longer to converge due to the more costly steps, but that each step tends to provide greater improvements. Indeed, we observe `SimplexWalk` dominating in the three scenarios with smallest depth and width in Fig. 6.

4.6 A Case Study on Adaptive Stepsize for $\texttt{PPGA}_{\text{LR}}$

To better understand why $\texttt{PPGA}_{\text{LR}}$ dominates in deeper instances, we present a case study on a randomly generated instance with $(n_0 = 1000, d = 6, w = 1000)$ and seed $s = 30$. We ran 1000 iterations of `PPGA` with five learning rates, $\gamma \in \{5, 50, 500, 5000, 50000\}$. We also ran 1000 iterations of $\texttt{PPGA}_{\text{LR}}$, but with only three different learning rates, $\gamma \in \{5, 500, 50000\}$. We use fewer learning rates with $\texttt{PPGA}_{\text{LR}}$ because there is a greater overlap in step sizes and objective values across learning rates in this case, making it harder to visually distinguish them. Figure 7 shows the influence of the learning rate γ on the step size over time. Figure 8 shows the influence of the learning rate γ on the solution value.

Our three main observations are the following:

- In `PPGA`, when we increase the learning rate γ, the step size changes proportionally. In $\texttt{PPGA}_{\text{LR}}$, on the other hand, the step size is less affected by the learning rate and only the size of the smallest steps are clipped.
- In `PPGA`, the range of step sizes for a specific learning rate is small. In $\texttt{PPGA}_{\text{LR}}$, on the other hand, the valve mechanism defined by Eq. (20) allows for a larger range of step sizes regardless of learning rate. That seems to lead to similar convergence for step sizes in different orders of magnitude.
- Since there is no mechanism designed for avoiding excessively large step sizes, both algorithms perform similarly worse when $\gamma = 50000$. We could potentially address this by designing another valve and truncating large steps.

Hence, `PPGA` is more sensitive to the learning rate and does not converge well with lower learning rates ($\gamma \in \{5, 50\}$). In turn, $\texttt{PPGA}_{\text{LR}}$ is more robust and converges well with lower learning rates in deep neural networks, as observed in Fig. 6.

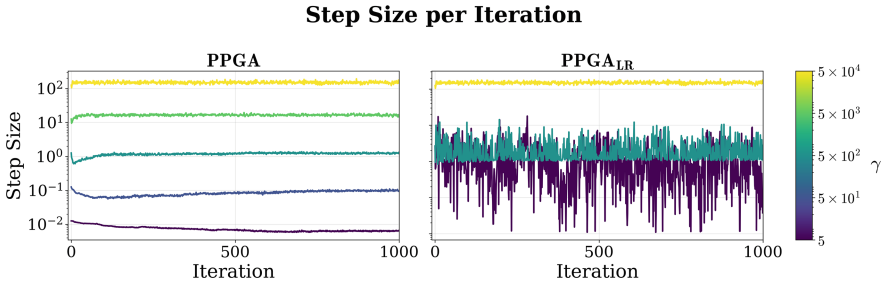


Fig. 7. Step size in log scale over iterations for `PPGA` (Left) and $\texttt{PPGA}_{\text{LR}}$ (Right) for each learning rate.

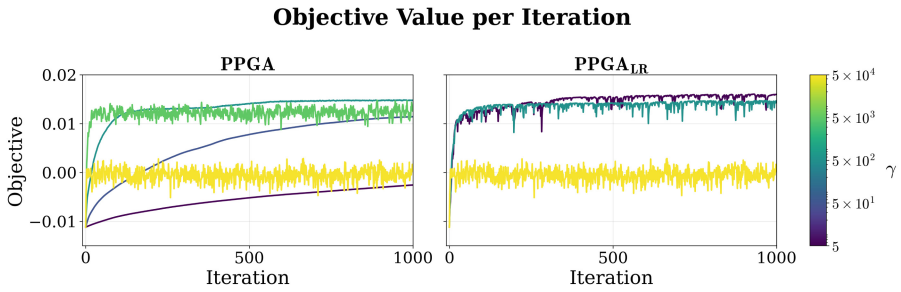


Fig. 8. Solution value over iterations for `PPGA` (Left) and $\texttt{PPGA}_{\text{LR}}$ (Right) for each learning rate.

5 Conclusion

In this paper, we proposed Gradient Walk: a new approach for local search based on gradient steps to optimize over the piecewise affine landscape of a trained ReLU network function within a polytope. We also proposed two algorithms for this approach, `PPGA` and $\texttt{PPGA}_{\text{LR}}$. `PPGA` is based on projected gradient steps and empirically-crafted perturbations. `PPGA` is designed to search for a feasible solution while avoiding entrapment in arbitrary local optima constructed by the special landscape of ReLU networks. $\texttt{PPGA}_{\text{LR}}$ extends `PPGA` by using adaptive gradient steps that stretch further when the step size is too small relative to the local affine landscape. $\texttt{PPGA}_{\text{LR}}$ is designed to circumvent the issue of diminishing step sizes caused by the combination of a misaligned learning rate with small gradients observed in deep ReLU networks. We report that $\texttt{PPGA}_{\text{LR}}$ has a better performance than `PPGA` in neural networks with larger dimensions, whereas both outperform the baseline projected gradient ascent algorithm `PGA` and compare favorably against the LP Walk algorithm `SimplexWalk`, which has previously shown better results than other alternatives in the benchmark setting [55].

In a nutshell, we double down on the trend of using specialized local search with lower per-iteration cost to find better solutions in constraint learning. We achieve that with gradient steps leveraging the local structure in ReLU networks.

Given the emergence of constraint learning frameworks [4,9,13,18,33,35,58] for applications in operations research, the development of new algorithms to timely find better solutions is of particular importance.

In future work, we intend to devise a variant of `PPGA` that is also robust against large learning rates, explore how other characteristics of the landscape of ReLU networks can be leveraged to produce better algorithms, apply the Gradient Walk approach to solve other types of constraint learning models, and explore its performance on specific families of functions approximated by neural networks.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Anderson, R., Huchette, J., Ma, W., Tjandraatmadja, C., Vielma, J.P.: Strong mixed-integer programming formulations for trained neural networks. Math. Program. (2020)
2. Arora, R., Basu, A., Mianjy, P., Mukherjee, A.: Understanding deep neural networks with rectified linear units. In: ICLR (2018)
3. Badilla, F., Goycoolea, M., Muñoz, G., Serra, T.: Computational tradeoffs of optimization-based bound tightening in ReLU networks. arXiv:2312.16699 (2023)
4. Bergman, D., Huang, T., Brooks, P., Lodi, A., Raghunathan, A.U.: JANOS: an integrated predictive and prescriptive modeling framework. INFORMS J. Comput. (2022)
5. Bonami, P., Lodi, A., Tramontani, A., Wiese, S.: On mathematical programming with indicator constraints. Math. Program. **151**(1), 191–223 (2015). https://doi.org/10.1007/s10107-015-0891-4
6. Botoeva, E., Kouvaros, P., Kronqvist, J., Lomuscio, A., Misener, R.: Efficient verification of (ReLU)-based neural networks via dependency analysis. In: AAAI (2020)
7. Cacciola, M., Frangioni, A., Lodi, A.: Structured pruning of neural networks for constraints learning. Oper. Res. Lett. (2024)
8. Cai, J., et al.: Getting away with more network pruning: from sparsity to geometry and linear regions. In: CPAIOR (2023)
9. Ceccon, F., et al.: OMLT: optimization & machine learning toolkit. J. Mach. Learn. Res. **23**(349), 1–8 (2022)
10. Cheng, C., Nührenberg, G., Ruess, H.: Maximum resilience of artificial neural networks. In: ATVA (2017)
11. Cybenko, G.: Approximation by superpositions of a sigmoidal function. Math. Control, Signals Syst. (1989)
12. Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. Math. Program. **91**(2), 201–213 (2002). https://doi.org/10.1007/s101070100263
13. Fajemisin, A., Maragno, D., den Hertog, D.: Optimization with constraint learning: a framework and survey. Eur. J. Oper. Res. (2023)
14. Fischetti, M., Jo, J.: Deep neural networks and mixed integer linear optimization. Constraints **23**(3), 296–309 (2018). https://doi.org/10.1007/s10601-018-9285-6
15. Funahashi, K.I.: On the approximate realization of continuous mappings by neural networks. Neural Networks (1989)
16. Grimstad, B., Andersson, H.: ReLU networks as surrogate models in mixed-integer linear programs. Comput. Chem. Eng. (2019)
17. Guo, X., Han, J., Tajrobehkar, M., Tang, W.: Escaping saddle points efficiently with occupation-time-adapted perturbations. arXiv preprint arXiv:2005.04507 (2020)
18. Gurobi Optimization: Gurobi Machine Learning. https://github.com/Gurobi/gurobi-machinelearning (2025). Accessed 09 Feb 2025
19. Gurobi Optimization, LLC: Gurobi Optimizer Reference Manual (2024). https://www.gurobi.com
20. Hanin, B., Sellke, M.: Approximating continuous functions by ReLU nets of minimal width. arXiv:1710.11278 (2017)
21. Hojny, C., Zhang, S., Campos, J.S., Misener, R.: Verifying message-passing neural networks via topology-based bounds tightening. In: ICML (2024)

22. Hornik, K., Stinchcombe, M., White, H.: Multilayer feedforward networks are universal approximators. Neural Networks (1989)
23. Huchette, J.: Advanced mixed-integer programming formulations: methodology, computation, and application. Ph.D. thesis, Massachusetts Institute of Technology (2018)
24. Huchette, J., Muñoz, G., Serra, T., Tsay, C.: When deep learning meets polyhedral theory: a survey. INFORMS J. Comput. (2026)
25. Huchette, J., Vielma, J.P.: Nonconvex piecewise linear functions: advanced formulations and simple modeling tools. Oper. Res. (2023)
26. Jin, C., Ge, R., Netrapalli, P., Kakade, S.M., Jordan, M.I.: How to escape saddle points efficiently. In: International Conference on Machine Learning, pp. 1724–1732. PMLR (2017)
27. Jin, C., Netrapalli, P., Jordan, M.I.: Accelerated gradient descent escapes saddle points faster than gradient descent. In: Conference On Learning Theory, pp. 1042–1085. PMLR (2018)
28. Kanamori, K., Takagi, T., Kobayashi, K., Ike, Y., Uemura, K., Arimura, H.: Ordered counterfactual explanation by mixed-integer linear optimization. In: AAAI (2021)
29. Kumar, A., Serra, T., Ramalingam, S.: Equivalent and approximate transformations of deep neural networks. arXiv:1905.11428 (2019)
30. Liu, C., Arnon, T., Lazarus, C., Strong, C., Barrett, C., Kochenderfer, M.J., et al.: Algorithms for verifying deep neural networks. Found. Trends® Optim. (2021)
31. Liu, X., Han, X., Zhang, N., Liu, Q.: Certified monotonic neural networks. In: NeurIPS, vol. 33 (2020)
32. Liu, X., Dvorkin, V.: Optimization over trained neural networks: difference-of-convex algorithm and application to data center scheduling. IEEE Control Syst. Lett. (2025)
33. Lueg, L., Grimstad, B., Mitsos, A., Schweidtmann, A.M.: reluMIP: open source tool for MILP optimization of ReLU neural networks (2021). https://doi.org/10.5281/zenodo.5601907, https://github.com/ChemEngAI/ReLU_ANN_MILP
34. Mangasarian, O.L., Rosen, J.B., Thompson, M.E.: Global minimization via piecewise-linear underestimation. J. Global Optim. (2005)
35. Maragno, D., Wiberg, H., Bertsimas, D., Birbil, S.I., Hertog, D.D., Fajemisin, A.: Mixed-integer optimization with constraint learning. Oper. Res. (2023)
36. McCormick, G.P.: Computability of global solutions to factorable nonconvex programs: part I — convex underestimating problems. Math. Program. (1976)
37. Misener, R., Floudas, C.A.: Piecewise-linear approximations of multidimensional functions. J. Optim. Theory Appl. (2010)
38. Perakis, G., Tsiourvas, A.: Optimizing objective functions from trained ReLU neural networks via sampling. arXiv:2205.14189 (2022)
39. Pham, H., Ren, A., Tahir, I., Tong, J., Serra, T.: Optimization over trained (and sparse) neural networks: a surrogate within a surrogate. In: CPAIOR (2026)
40. Plate, C., Hahn, M., Klimek, A., Ganzer, C., Sundmacher, K., Sager, S.: An analysis of optimization problems involving ReLU neural networks. arXiv:2502.03016 (2025)
41. Polyak, B.T., Levitin, E.: Constrained minimization methods. USSR Comput. Math. Math. Phys. **6**(5), 1–50 (1966)
42. Rosen, J.B.: The gradient projection method for nonlinear programming. part i. linear constraints. J. Soc. Ind. Appl. Math. **8**(1), 181–217 (1960)
43. Rössig, A., Petkovic, M.: Advances in verification of ReLU neural networks. J. Global Optim. (2021)

44. Say, B., Wu, G., Zhou, Y.Q., Sanner, S.: Nonlinear hybrid planning with deep net learned transition models and mixed-integer linear programming. In: IJCAI (2017)
45. Schweidtmann, A.M., Mitsos, A.: Deterministic global optimization with artificial neural networks embedded. J. Optim. Theory Appl. (2019)
46. Serra, T., Ramalingam, S.: Empirical bounds on linear regions of deep rectifier networks. In: AAAI (2020)
47. Serra, T., Tjandraatmadja, C., Ramalingam, S.: Bounding and counting linear regions of deep neural networks. In: ICML (2018)
48. Serra, T., Yu, X., Kumar, A., Ramalingam, S.: Scaling up exact neural network compression by ReLU stability. In: NeurIPS (2021)
49. Serra, T., Kumar, A., Ramalingam, S.: Lossless compression of deep neural networks. In: CPAIOR (2020)
50. Shi, C., Emadikhiav, M., Lozano, L., Bergman, D.: Constraint learning to define trust regions in optimization over pre-trained predictive models. INFORMS J. Comput. (2024)
51. Sosnin, P., Tsay, C.: Scaling mixed-integer programming for certification of neural network controllers using bounds tightening. In: CDC (2024)
52. Strong, C.A., et al.: Global optimization of objective functions represented by ReLU networks. Mach. Learn. (2021)
53. Tits, A.L., Yang, Y.: Globally convergent algorithms for robust pole assignment by state feedback. IEEE Trans. Automat. Control **41**(10), 1432–1452 (1996). https://doi.org/10.1109/9.539425
54. Tjeng, V., Xiao, K., Tedrake, R.: Evaluating robustness of neural networks with mixed integer programming. In: ICLR (2019)
55. Tong, J., Cai, J., Serra, T.: Optimization over trained neural networks: taking a relaxing walk. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, pp. 221–233. Springer (2024). https://doi.org/10.1007/978-3-031-60599-4_14
56. Tsay, C., Kronqvist, J., Thebelt, A., Misener, R.: Partition-based formulations for mixed-integer optimization of trained ReLU neural networks. In: NeurIPS (2021)
57. Tsiourvas, A., Sun, W., Perakis, G.: Manifold-aligned counterfactual explanations for neural networks. In: AISTATS (2024)
58. Turner, M., Chmiela, A., Koch, T., Winkler, M.: PySCIPOpt-ML: embedding trained machine learning models into mixed-integer programs. In: CPAIOR (2020)
59. Vahedi, A.M., Ilies, H.T.: SPGD: steepest perturbed gradient descent optimization. arXiv preprint arXiv:2411.04946 (2024)
60. Vielma, J.P., Ahmed, S., Nemhauser, G.: Mixed-integer models for nonseparable piecewise-linear optimization: Unifying framework and extensions. Oper. Res. (2010)
61. Xiao, K.Y., Tjeng, V., Shafiullah, N.M., Madry, A.: Training for faster adversarial robustness verification via inducing ReLU stability. In: ICLR (2019)
62. Yarotsky, D.: Error bounds for approximations with deep ReLU networks. Neural Networks (2017)
63. Zhao, H., Hijazi, H., Jones, H., Moore, J., Tanneau, M., Hentenryck, P.V.: Bound tightening using rolling-horizon decomposition for neural network verification. In: CPAIOR (2024)
64. Zhu, Y., Burer, S.: An extended validity domain for constraint learning. arXiv:2406.10065 (2024)